

Practice

Vehicle Routing Problem

Team:

Santiago Betancur
Yasir Blandón Varela
Daniel Felipe Arango

Academic Instructor:

Alejandro Arenas Vasco

EAFIT
Department of Computer Science and Systems
Numerical Analysis

Medellín - Antioquia
May 29, 2025

Algorithm Description

The implemented algorithm is a version of the nearest neighbor approach. For this project, a probabilistic strategy was used to solve the vehicle routing problem with limited vehicle capacity. Its goal is to construct routes from a depot to multiple clients, minimizing the total traveled distance while allowing some randomness in the node selection to explore different solutions.

Each route starts at the depot and attempts to add customers whose demand does not exceed the vehicle capacity, which is 12 customers. Unlike the classical method, this algorithm selects the next customer based on a probability inversely proportional to the distance—favoring closer nodes, but not strictly.

This approach allows better exploration of the solution space and can avoid poor local solutions that are common in fully deterministic methods.

Algorithm Pseudocode

Algorithm 1: Route Construction using Probabilistic Nearest Neighbor

Input:

D : distance matrix between points
 $clients$: set of nodes to visit (excluding the depot)
 Q : maximum vehicle capacity
 $demands$: demand array for each client

Output: List of routes, each containing a subset of clients

```
1  $routes \leftarrow$  empty list;  
2  $unvisited \leftarrow$  copy of  $clients$ ;  
3 while  $unvisited$  is not empty do  
4    $route \leftarrow$  empty list;  
5    $load \leftarrow 0$ ;  
6    $current \leftarrow 0$  // starts at the depot  
7   while there exists client  $c$  in  $unvisited$  such that  $load + demands[c] \leq Q$   
   do  
8      $neighbors \leftarrow$  feasible clients from  $current$ ;  
9     Compute probabilities  $\propto \frac{1}{distance(current,c)}$  for each  $c$  in  $neighbors$ ;  
10     $next \leftarrow$  randomly chosen client based on probabilities;  
11    Add  $next$  to  $route$ ;  
12     $load \leftarrow load + demands[next]$ ;  
13    Remove  $next$  from  $unvisited$ ;  
14     $current \leftarrow next$ ;  
15  Insert depot at the beginning of  $route$ ;  
16  Insert depot at the end of  $route$ ;  
17  Add  $route$  to  $routes$ ;  
18 return  $routes$ ;
```
